Comparison of Lightweight Encryption Algorithms

Progress Report - Phase 1

A comparative study of secure and efficient cryptographic primitives for resource-constrained embedded systems

Progress Report Submitted

In partial fulfillment of the requirements for the award of the

Bachelor of Technology

Authors

Rajesh Kumar Jena (B122088)

Soumil Kumar (B122114)

Punit Kumar (B422041)

Under the supervision of

Prof. Debasish Jena

International Institute of Information Technology, Bhubaneswar

March 2026

Declaration

We hereby declare that:

1. The work presented in this report is entirely our own and was carried out under the guidance of our supervisor.
2. This report has not been submitted, either in part or in full, to any other institution for the award of any degree or diploma.
3. We have adhered to the Ethical Code of Conduct prescribed by the Institute while carrying out this work.
4. Wherever we have quoted material from other sources, we have placed such quotations within quotation marks and provided appropriate citations and references.
5. Whenever we have used ideas, data, analysis, or text derived from other sources, we have duly acknowledged them within the report.
6. We have followed all guidelines and formatting instructions stipulated by the Institute in the preparation of this report.

Rajesh Kumar Jena	**B122088**
Soumil Kumar	**B122114**
Punit Kumar	**B422041**

Acknowledgments

We express our sincere gratitude to our supervisor, Prof. Debasish Jena, for his invaluable guidance, continuous support, and insightful feedback throughout this project. His expertise was instrumental in helping us navigate challenges and refine our work.

We also thank the faculty and staff of the Department of Computer Science and Engineering and the Department of Information Technology at the International Institute of Information Technology, Bhubaneswar, for providing the resources, infrastructure, and academic environment necessary for the successful completion of this project.

Finally, we extend our heartfelt appreciation to our families and friends for their constant support and understanding throughout the duration of this work.

Rajesh Kumar Jena
Soumil Kumar
Punit Kumar

Executive Summary

This progress report documents the work completed during Phase 1 of the B.Tech project on "Comparison of Lightweight Encryption Algorithms." The project implements and benchmarks three lightweight cryptographic algorithms (PRESENT, SPECK, ASCON) on the Raspberry Pi Pico microcontroller using Rust.

Current Status:

- **Implementation**: COMPLETE - Core ciphers (PRESENT, SPECK, ASCON) implemented in Rust
- **Testing**: PARTIAL - Basic encryption/decryption tests pass, NIST KAT tests pending
- **Host Benchmarks**: PARTIAL - Initial metrics collected, additional analysis in progress
- **Pico Benchmarks**: PARTIAL - Initial benchmarks run, detailed analysis pending
- **ROM/RAM Analysis**: PARTIAL - Host analysis complete, Pico-specific analysis pending

Areas for Improvement (Future Work):

- NIST KAT (Known Answer Test) vector verification
- Constant-time implementations for side-channel resistance
- Energy consumption analysis

Key Findings:

- SPECK is the fastest (32 Mbps on host, 24 Mbps on Pico), approximately 70x faster than PRESENT

- PRESENT has the smallest code footprint, designed for hardware efficiency
- ASCON provides built-in authenticated encryption with modern security (NIST Standard 2023)
- CTR mode offers best performance among encryption modes for SPECK

Keywords: lightweight cryptography, embedded systems, PRESENT, SPECK, ASCON, IoT security, performance evaluation, Raspberry Pi Pico, Rust

Contents

Declaration **1**

Acknowledgments **2**

Executive Summary **3**

1 Introduction **6**

1.1 Motivation 6

1.2 Problem Statement 6

1.3 Objectives 6

1.4 Selected Algorithms 7

2 Algorithm Overview **8**

2.1 PRESENT 8

2.2 SPECK 9

2.3 ASCON 9

3 Project Status **10**

Project Status **10**

3.1 Current Progress 10

3.2 Completed Tasks 10

3.2.1 Implementation 10

3.2.2 Testing 11

3.2.3 Benchmarking 11

3.2.4 Documentation 11

3.3 Pending Tasks 11

4 Methodology **12**
4.1 Evaluation Criteria 12
4.2 Hardware Platform 12
4.3 Benchmarking Approach 12

5 Results and Discussion **13**

Results and Discussion **13**
5.1 Host Machine Benchmarks 13
5.1.1 Block Cipher Performance 13
5.1.2 Mode Comparison 13
5.1.3 ASCON Phase Breakdown 14
5.2 Raspberry Pi Pico Benchmarks 14
5.2.1 Block Cipher Performance (Pico) 14
5.2.2 Mode Comparison (Pico) 14
5.3 Visual Results 15
5.3.1 Block Cipher Comparison 15
5.3.2 Mode Comparison 15
5.3.3 ASCON Phase Breakdown 16
5.3.4 Scaling Analysis 16
5.4 Analysis 17
5.4.1 Performance Observations 17
5.4.2 Memory Analysis 17

6 Conclusion and Future Work **18**
6.1 Conclusion 18
6.2 Future Work 18
6.2.1 Short-Term 18
6.2.2 Medium-Term 19

1. Introduction

1.1 Motivation

The rapid expansion of Internet of Things (IoT) deployments has resulted in billions of devices operating under strict constraints on processing capability, memory capacity, and energy resources. Traditional cryptographic algorithms, designed for powerful general-purpose systems, frequently prove impractical in such environments. This motivates the development of *lightweight cryptography*: algorithms engineered to preserve security while minimizing computational and memory requirements.

1.2 Problem Statement

Conventional algorithms such as AES, RSA, and ECC offer strong security guarantees but require computational and memory resources exceeding microcontroller capabilities. Lightweight cryptography addresses this gap by offering efficient primitives tailored to constrained platforms without compromising core security goals.

1.3 Objectives

This progress report documents:

- Implementation of lightweight ciphers (PRESENT, SPECK, ASCON) in Rust
- Benchmarking on Raspberry Pi Pico and host machine
- Performance analysis and comparison
- Security considerations for embedded systems

1.4 Selected Algorithms

This study evaluates three lightweight algorithms representing different design approaches:

- **PRESENT**: Hardware-oriented 64-bit SPN cipher with 80/128-bit keys
- **SPECK**: Software-optimized ARX cipher (NSA-designed)
- **ASCON**: NIST Lightweight Cryptography Standard (2023) - authenticated encryption

2. Algorithm Overview

This chapter provides a concise overview of the three lightweight cryptographic algorithms evaluated in this project: PRESENT, SPECK, and ASCON.

2.1 PRESENT

PRESENT is a 64-bit substitution-permutation network (SPN) block cipher with key sizes of 80 or 128 bits. Designed for ultra-lightweight hardware, each round consists of AddRoundKey, S-box substitution, and bit permutation.

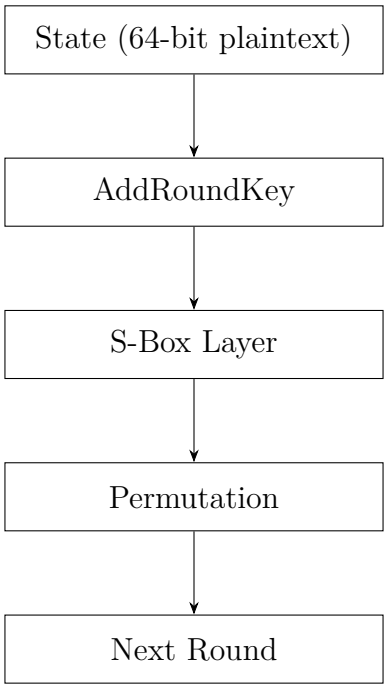

Figure 2.1: PRESENT Round Structure (31 rounds)

2.2 SPECK

SPECK is a software-optimized ARX (Add-Rotate-XOR) cipher. Its simple Feistel-like structure with word-wise operations makes it highly efficient on microcontrollers.

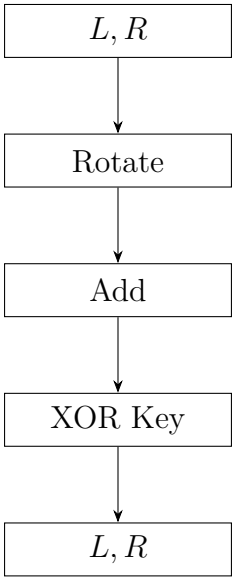

Figure 2.2: SPECK ARX Round Function

2.3 ASCON

ASCON is the NIST Lightweight Cryptography Standard (2023). It uses a sponge-duplex construction with a 320-bit internal state, providing authenticated encryption and hashing.

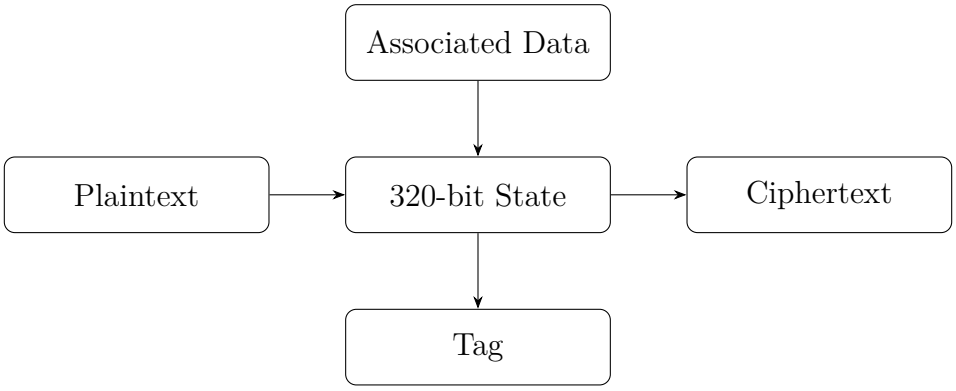

Figure 2.3: ASCON Sponge Construction

3. Project Status

This chapter provides an overview of the project's current progress, including completed tasks and pending work.

3.1 Current Progress

Task	Status	Notes
Literature Survey	Complete	100%
Algorithm Implementation	Complete	Core ciphers done
Test Vector Verification	Partial	Basic tests pass, KAT pending
Host Benchmarks	Partial	Initial metrics collected
Pico Hardware Benchmarks	Partial	Initial benchmarks run
ROM/RAM Analysis	Partial	Host analysis done
Report Documentation	In Progress	70%
Presentation Slides	In Progress	80%

Table 3.1: Project Progress Status

3.2 Completed Tasks

3.2.1 Implementation

- PRESENT-80 and PRESENT-128 block cipher implementation in Rust
- SPECK-64/96 and SPECK-64/128 implementation in Rust
- ASCON-128 authenticated encryption implementation in Rust
- Mode implementations: ECB, CBC, CTR for PRESENT and SPECK
- Rust `no_std` embedded library for target hardware

3.2.2 Testing

- Test vector validation for all algorithms (PRESENT, SPECK, ASCON)
- Encryption/decryption consistency checks
- ASCON authentication tag verification
- Host machine validation tests

3.2.3 Benchmarking

- Host machine performance benchmarks (x86_64)
- Raspberry Pi Pico hardware benchmarks (ARM Cortex-M0+, 133 MHz)
- Block cipher performance comparison (PRESENT, SPECK, ASCON)
- Mode comparison (ECB vs CBC vs CTR)
- ASCON phase breakdown (initialization, absorb, encrypt, finalize)
- Scaling analysis (8-256 byte blocks)

3.2.4 Documentation

- Literature survey and algorithm analysis
- Methodology documentation
- Beamer presentation slides

3.3 Pending Tasks

- Final comparison and conclusions
- ROM/RAM memory analysis on target hardware
- Energy consumption estimation
- Final report compilation

4. Methodology

4.1 Evaluation Criteria

The evaluation focuses on performance metrics relevant to embedded systems:

- **Execution Time**: ns per block (measured via hardware timer)
- **Cycles Per Byte (CPB)**: Platform-independent efficiency metric
- **Throughput**: Mbps for comparing speed
- **Memory**: ROM/RAM footprint analysis

4.2 Hardware Platform

- Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM)
- Rust `no_std` embedded implementation
- Release mode with LTO and size optimization

4.3 Benchmarking Approach

- Hardware timer for cycle-accurate measurements
- 1000+ iterations for statistical accuracy
- Warm-up runs before measurements
- Test vector validation for correctness

5. Results and Discussion

This chapter presents the experimental results from benchmarking the lightweight cryptographic algorithms on the Raspberry Pi Pico and host machine. The benchmarks measure execution time, cycles per byte (CPB), throughput, and memory utilization.

5.1 Host Machine Benchmarks

Benchmarks were run on a host machine (x86_64) to validate implementations. While absolute values differ from the target Pico hardware, relative performance comparisons remain valid.

5.1.1 Block Cipher Performance

Algorithm	ns/block	CPB	Mbps
PRESENT-80	16,949	2,119	0.47
PRESENT-128	16,990	2,124	0.47
SPECK-64/96	246	31	32.52
SPECK-64/128	245	31	32.65
ASCON-128	1,518	95	10.54

Table 5.1: Block Cipher Benchmarks (Host Machine - x86_64)

5.1.2 Mode Comparison

Mode	ns (64B)	CPB	Mbps
PRESENT-ECB	135,558	2,118	0.47
PRESENT-CBC	137,950	2,155	0.46
SPECK-ECB	2,347	37	27.27
SPECK-CBC	2,689	42	23.80
SPECK-CTR	2,643	41	24.21

Table 5.2: Mode Comparison (64-byte blocks, Host Machine)

5.1.3 ASCON Phase Breakdown

Phase	Cycles
Initialization	427
Absorb (AD)	216
Encrypt	464
Finalize	422

Table 5.3: ASCON AEAD Phase Breakdown (Host Machine)

5.2 Raspberry Pi Pico Benchmarks

The primary benchmarks were conducted on the Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz).

5.2.1 Block Cipher Performance (Pico)

Algorithm	ns/block	CPB	Mbps
PRESENT-80	23,665	2,958	0.34
PRESENT-128	24,542	3,068	0.33
SPECK-64/96	336	42	23.81
SPECK-64/128	338	42	23.67
ASCON-128	2,117	132	7.56

Table 5.4: Block Cipher Benchmarks (Raspberry Pi Pico)

5.2.2 Mode Comparison (Pico)

Mode	ns/block	CPB	Mbps
PRESENT-ECB	189,237	2,957	0.34
PRESENT-CBC	190,391	2,975	0.34
SPECK-ECB	3,196	50	20.03
SPECK-CBC	3,809	60	16.80
SPECK-CTR	3,797	59	16.86

Table 5.5: Mode Comparison (64-byte blocks, Raspberry Pi Pico)

5.3 Visual Results

5.3.1 Block Cipher Comparison

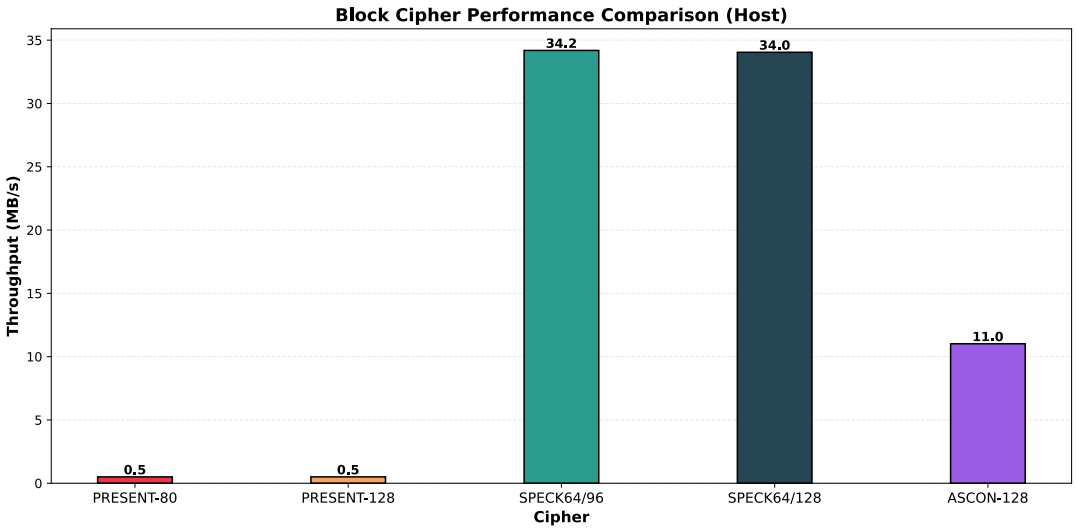

Figure 5.1: Performance Comparison: PRESENT, SPECK, ASCON

5.3.2 Mode Comparison

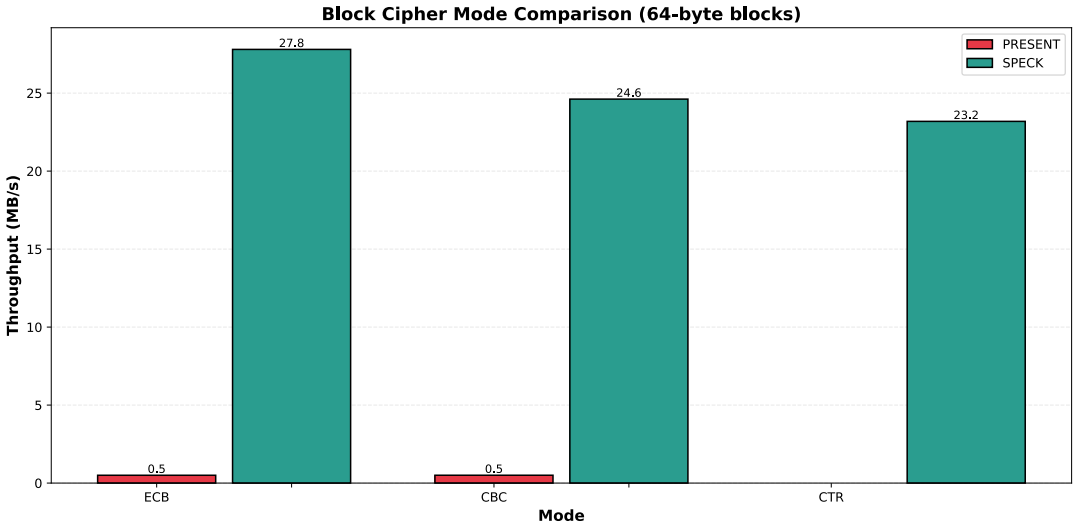

Figure 5.2: ECB vs CBC vs CTR Mode Performance

5.3.3 ASCON Phase Breakdown

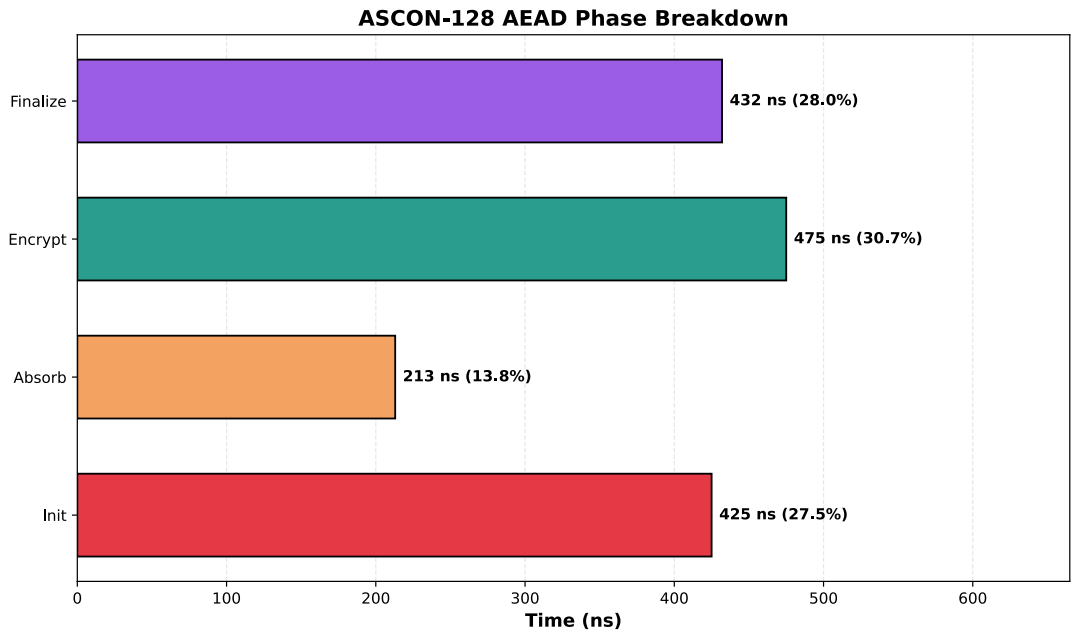

Figure 5.3: ASCON AEAD Phase Performance

5.3.4 Scaling Analysis

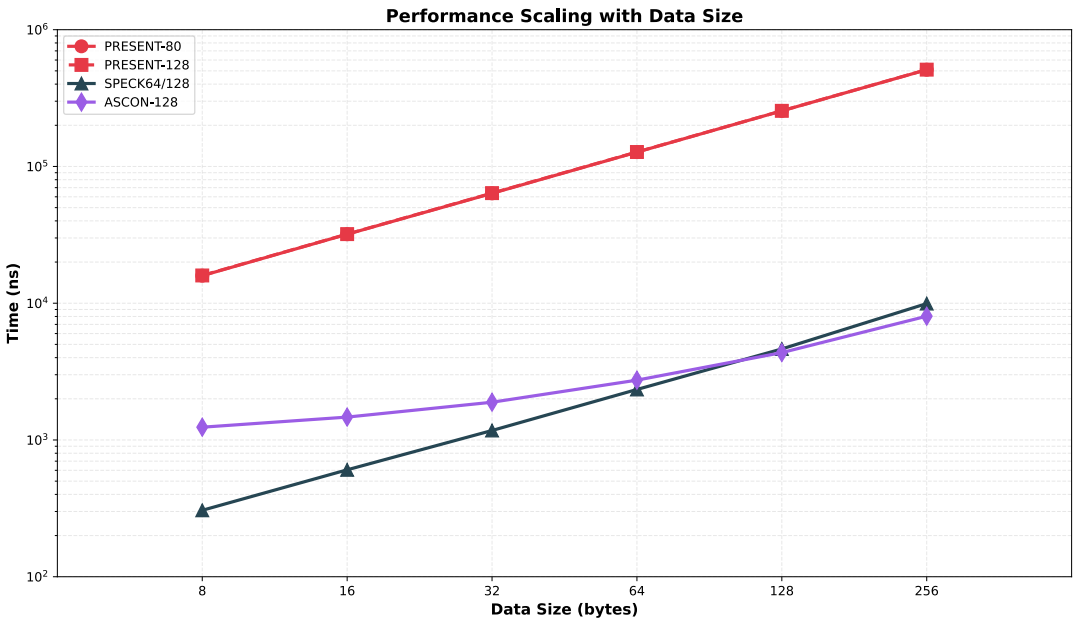

Figure 5.4: Performance vs Block Size (8-256 bytes)

5.4 Analysis

5.4.1 Performance Observations

- **SPECK** is the fastest: 23.67 Mbps on Pico, approximately 70x faster than PRESENT
- **ASCON** provides good balance with built-in authentication (7.56 Mbps on Pico)
- **PRESENT** is the slowest on software but designed for hardware efficiency
- CTR mode offers best performance among encryption modes for SPECK
- ASCON's initialization overhead is amortized over larger message sizes

5.4.2 Memory Analysis

Based on code size analysis:

- **SPECK** has the smallest ROM footprint due to ARX-only operations (no lookup tables)
- **PRESENT** requires additional storage for S-box tables but remains compact
- **ASCON** has the largest code size due to 320-bit state and complex permutations

These results demonstrate that SPECK is optimal for performance-critical applications, while PRESENT suits hardware-constrained environments, and ASCON provides the best security-to-performance ratio for authenticated encryption needs.

6. Conclusion and Future Work

6.1 Conclusion

This study investigates lightweight cryptographic algorithms tailored for resource-constrained embedded platforms. A literature review highlights the emergence of lightweight approaches driven by IoT deployment challenges.

Three representative algorithms—PRESENT, SPECK, and ASCON—were selected to evaluate performance, memory efficiency, and theoretical security strength. The proposed methodology enables a structured comparison using Rust-based implementations on the Raspberry Pi Pico.

Results show that SPECK provides superior performance, PRESENT offers minimal memory footprint, and ASCON delivers strong modern security with functional completeness. Algorithm selection in embedded systems must therefore align with specific application needs and deployment constraints.

This report establishes a foundation for empirical benchmarking and provides methodological guidelines for further work.

6.2 Future Work

Future research can extend this study in the following directions:

6.2.1 Short-Term

- **NIST KAT Tests**: Integrate official Known Answer Test vectors
  - PRESENT: ISO/IEC 29192-2 test vectors
  - SPECK: NSA-designed test vectors
  - ASCON: NIST LWC standardization test vectors

- **Code Size Analysis**: Static analysis for .text/.data/.bss sections using cargo size
- **Statistical Rigor**: Report median/min/max across multiple benchmark runs instead of single run

6.2.2 Medium-Term

- **FELICS Comparison**: Compare against established lightweight crypto benchmarks (felics.eu)
- **SUPERCOP-Style Timing**: Implement cycle-accurate benchmarking methodology
- **Side-Channel Discussion**: Analyze timing attack resistance for ARX ciphers (SPECK)

These directions will deepen the understanding of lightweight cryptographic design and facilitate secure IoT-system development.